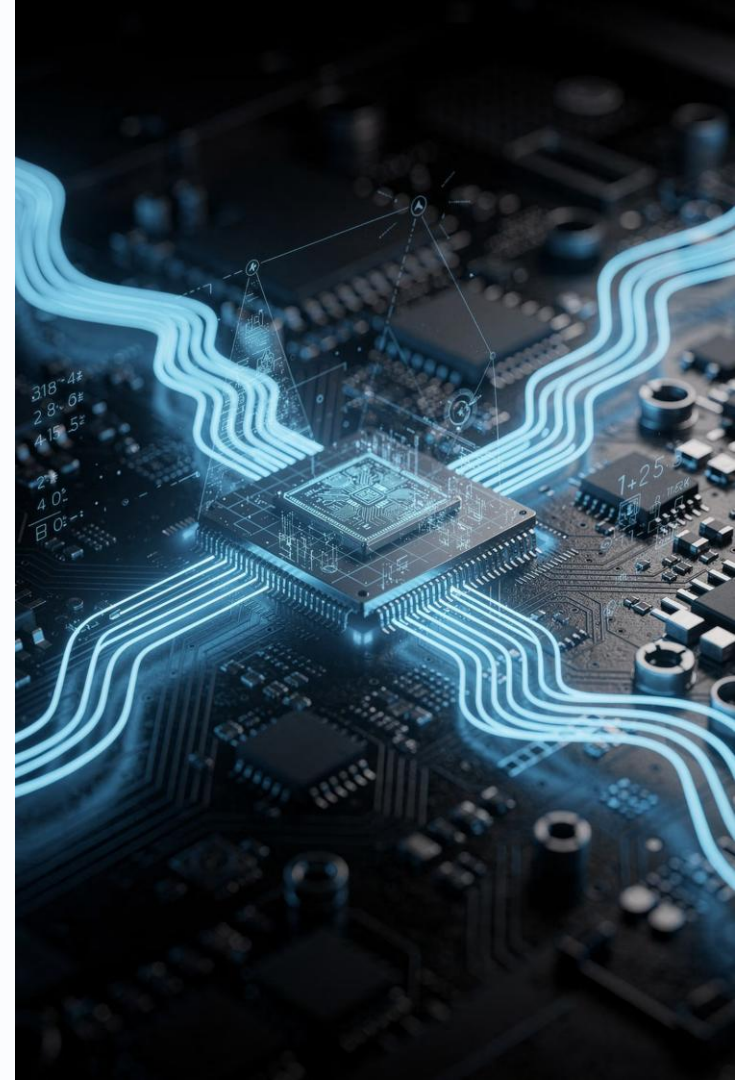


Algoritmos de Ordenamiento

Análisis de Eficiencia y Aplicaciones Reales

Introducción a la Computación II



Contenido de la Sesión

¿Qué veremos hoy?

Una guía completa a través del mundo del ordenamiento de datos: desde los conceptos fundamentales hasta la selección inteligente del algoritmo correcto para cada problema.

01

Fundamentos del Ordenamiento

Qué es, para qué sirve y ejemplos del mundo real

03

Los 4 Algoritmos Clave

Bubble, Insertion, Selection y Quick Sort

02

Notación Big O

Medir eficiencia: tiempo, memoria y complejidad

04

Comparación y Casos de Uso

¿Cuándo usar cada uno? Tabla comparativa y escenarios reales



¿Qué es un Algoritmo de Ordenamiento?

Un **algoritmo de ordenamiento** es un conjunto de instrucciones que reorganiza una colección de datos en un orden definido —ascendente, descendente o según un criterio— de manera sistemática y predecible.



Bases de Datos

Recuperación rápida de registros ordenados por fecha, nombre o prioridad



Motores de Búsqueda

Resultados ordenados por relevancia, popularidad o fecha de publicación



E-Commerce

Productos ordenados por precio, calificación o disponibilidad en tiempo real



Redes y Sistemas

Enrutamiento de paquetes y priorización de procesos en sistemas operativos

Análisis de Eficiencia de Algoritmos

La **eficiencia computacional** determina qué tan bien un algoritmo aprovecha los recursos disponibles. Evaluar un algoritmo implica medir dos dimensiones fundamentales: el tiempo de ejecución y el uso de memoria, ambas expresadas en función del tamaño de la entrada **n**.

Notación Big O

Expresa el comportamiento en el **peor caso** conforme crece **n**. Nos dice cómo escala el algoritmo, no cuántos segundos tarda en una máquina específica.

- **$O(1)$** – Tiempo constante
- **$O(n)$** – Lineal: crece proporcional a **n**
- **$O(n \log n)$** – Óptimo para ordenamiento comparativo
- **$O(n^2)$** – Cuadrático: costoso para grandes volúmenes

¿Por qué importa?

Un algoritmo $O(n^2)$ con $n = 1,000,000$ ejecuta **1 billón de operaciones**.
Uno $O(n \log n)$ ejecuta apenas **20 millones**. La diferencia puede ser de horas versus milisegundos en producción.



La eficiencia no es un lujo académico: es un factor crítico en sistemas reales con millones de usuarios.

Bubble Sort — Método Burbuja

Compara pares de elementos adyacentes e intercambia los que están en orden incorrecto. Los elementos más grandes "burbujean" hacia el final en cada pasada.

Pseudocódigo

```
para i desde 0 hasta n-1:  
  para j desde 0 hasta n-i-2:  
    si arr[j] > arr[j+1]:  
      intercambiar(arr[j], arr[j+1])
```

Ejemplo visual: [5, 3, 8, 1, 2]

Pasada 1: [3,5,8,1,2]→[3,5,1,8,2]→[3,5,1,2,8]

Pasada 2: [3,1,5,2,8]→[3,1,2,5,8]

Resultado: [1,2,3,5,8] ✓

Complejidad

Mejor

$O(n)$

Promedio

$O(n^2)$

Peor

$O(n^2)$

Memoria

$O(1)$



Ineficiente para conjuntos grandes. Útil solo con datos casi ordenados o fines pedagógicos.



Insertion Sort – Método de Inserción

Construye la lista ordenada de izquierda a derecha, tomando cada elemento y **insertándolo en su posición correcta** dentro de la sublista ya ordenada. Imita la manera natural en que los humanos ordenamos naipes.

Pseudocódigo

```
para i desde 1 hasta n-1:  
    clave = arr[i]  
    j = i - 1  
    mientras j >= 0 y arr[j] > clave:  
        arr[j+1] = arr[j]  
        j = j - 1  
    arr[j+1] = clave
```

Ejemplo: [5, 3, 8, 1]

i=1: [3,5,8,1] → i=2: [3,5,8,1] → i=3: [1,3,5,8]

Complejidad

Mejor

$O(n)$

Promedio

$O(n^2)$

Peor

$O(n^2)$

Memoria

$O(1)$



Excelente para arreglos pequeños o casi ordenados. Eficiente en tiempo real (streaming de datos).





Selection Sort – Método de Selección

En cada pasada, **busca el elemento mínimo** del subarray no ordenado y lo coloca en su posición definitiva. Divide el arreglo en dos partes: la zona ordenada (izquierda) y la zona no ordenada (derecha).

Pseudocódigo

```
para i desde 0 hasta n-2:  
    min_idx = i  
    para j desde i+1 hasta n-1:  
        si arr[j] < arr[min_idx]:  
            min_idx = j  
    intercambiar(arr[i], arr[min_idx])
```

Ejemplo: [64, 25, 12, 22]

Pasada 1: min=12 → [12,25,64,22]

Pasada 2: min=22 → [12,22,64,25]

Pasada 3: min=25 → [12,22,25,64] ✓

Complejidad

Mejor

$O(n^2)$

Promedio

$O(n^2)$

Peor

$O(n^2)$

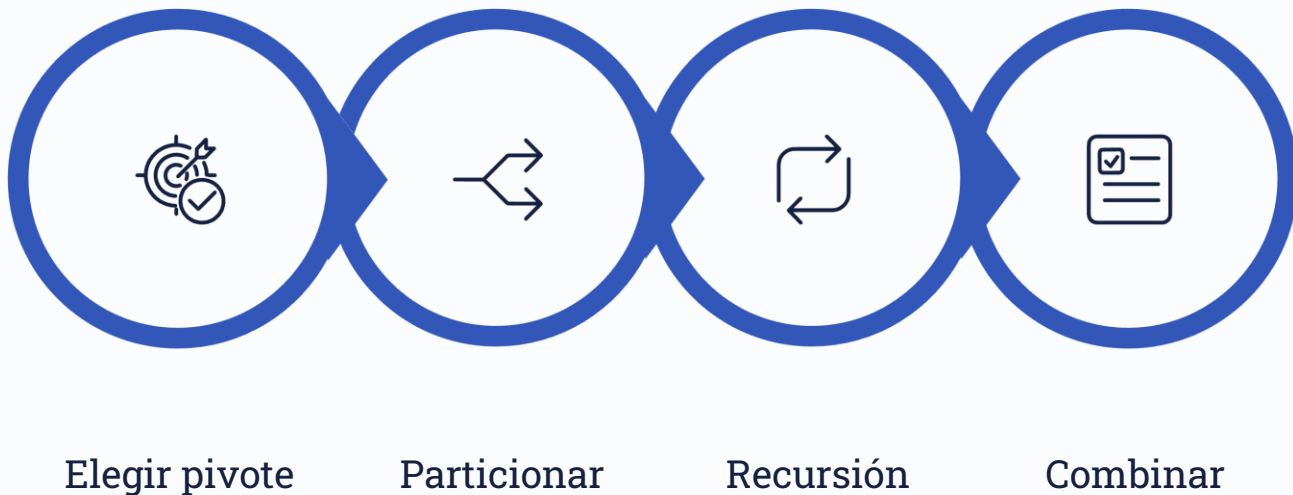
Memoria

$O(1)$

- ☐ Hace siempre $O(n^2)$ comparaciones, pero minimiza el número de intercambios: útil cuando escribir en memoria es costoso.

Quick Sort – Divide y Vencerás

El algoritmo más eficiente en la práctica. Selecciona un **pivote**, particiona el arreglo en elementos menores y mayores que el pivote, y aplica el mismo proceso recursivamente a cada partición.



La clave del poder de Quick Sort es la partición eficiente: con un buen pivote, divide el problema en mitades similares, logrando $O(n \log n)$ en el caso promedio. Su peor caso $O(n^2)$ ocurre con pivotes extremos (arreglos ya ordenados).

Pseudocódigo y Ejemplo Visual

Pseudocódigo

```
quicksort(arr, bajo, alto):  
    si bajo < alto:  
        pivote_idx = particionar(arr, bajo, alto)  
        quicksort(arr, bajo, pivote_idx - 1)  
        quicksort(arr, pivote_idx + 1, alto)  
  
particionar(arr, bajo, alto):  
    pivote = arr[alto]  
    i = bajo - 1  
    para j desde bajo hasta alto-1:  
        si arr[j] <= pivote:  
            i++  
            intercambiar(arr[i], arr[j])  
    intercambiar(arr[i+1], arr[alto])  
    retornar i + 1
```

Ejemplo: [10, 80, 30, 90, 40, 50, 70]

Pivote = 70

Partición: [10,30,40,50] | 70 | [80,90]

Recursión izquierda (pivote=50):

[10,30,40] | 50

Resultado final:

[10,30,40,50,70,80,90] ✓

Mejor

$O(n \log n)$

Promedio

$O(n \log n)$

Peor

$O(n^2)$

Memoria

$O(\log n)$

Tabla Comparativa de Eficiencia

La siguiente tabla resume la complejidad temporal y espacial de los cuatro algoritmos. Comprender estas diferencias es fundamental para tomar decisiones de diseño correctas en proyectos reales.

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Memoria	Estable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Sí
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Sí
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No

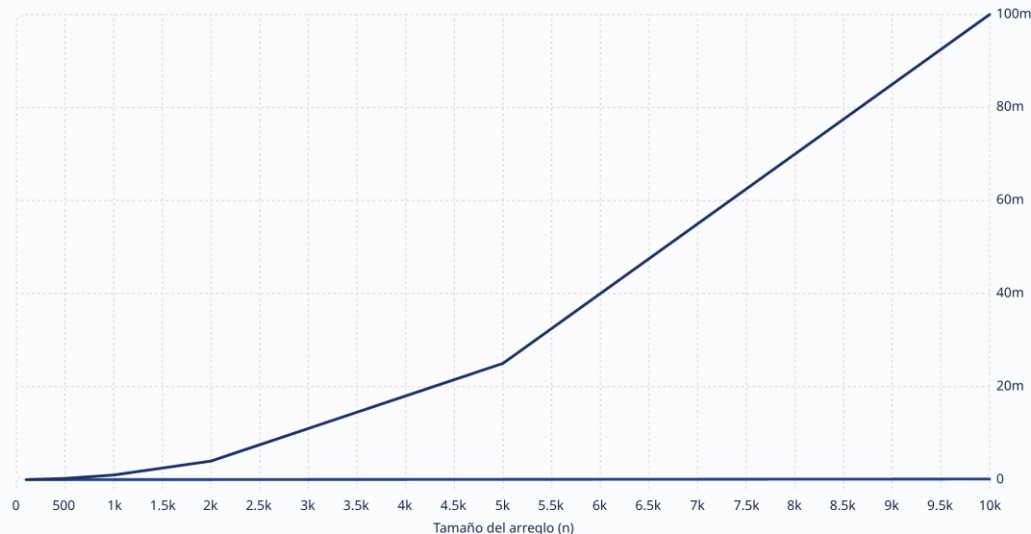
 Un algoritmo **estable** preserva el orden relativo de elementos con igual clave. Esencial en ordenamientos encadenados (e.g., ordenar por apellido y luego por nombre).

Visualización de Rendimiento Comparativo

A medida que el tamaño del conjunto de datos crece, la diferencia entre $O(n^2)$ y $O(n \log n)$ se vuelve drástica. Quick Sort maneja millones de elementos donde Bubble Sort se quedaría procesando durante horas.

Quick Sort $O(n \log n)$

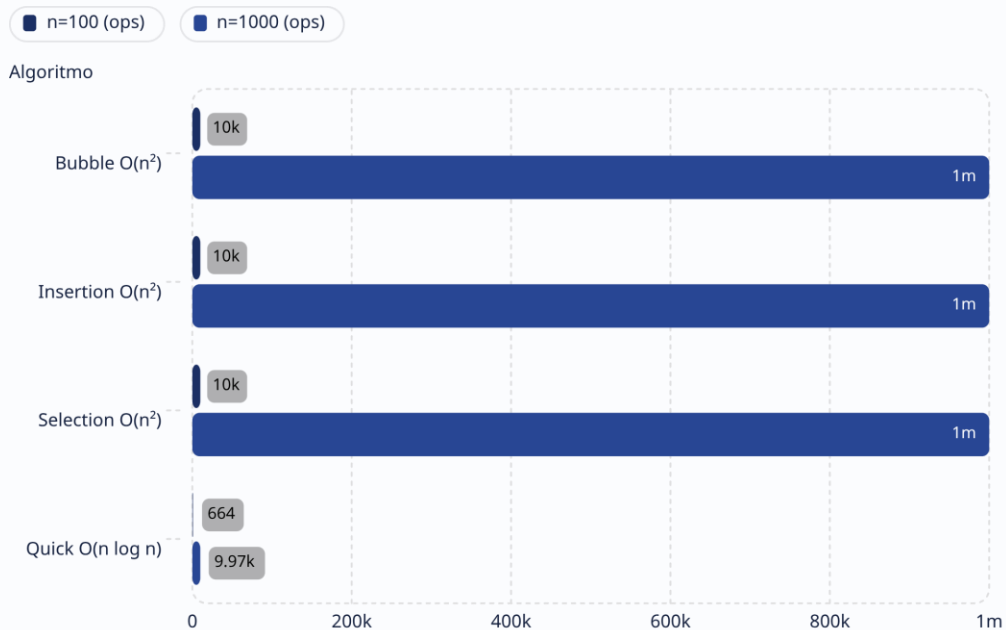
Bubble/Insertion/Selection $O(n^2)$



La brecha entre ambas curvas es exponencial: con $n=10,000$, $O(n^2)$ requiere **100 millones de operaciones** frente a las **~133 mil** de Quick Sort. En sistemas de producción con millones de registros, esta diferencia equivale a horas de cómputo versus milisegundos.

¿Cómo escalan los algoritmos?

Operaciones por Tamaño de Entrada



Lectura del gráfico

Con **$n=100$** , todos los algoritmos parecen similares — la diferencia no es perceptible para el usuario. La trampa está en escalar.

Con **$n=1,000$** , Quick Sort ejecuta 100 veces menos operaciones que los algoritmos cuadráticos.



Nunca evalúes un algoritmo solo con datos pequeños. Prueba siempre con conjuntos representativos del caso de uso real.

Capítulo 4 — Aplicaciones Prácticas

¿Cuándo Usar Cada Algoritmo?

Elegir el algoritmo correcto no es solo cuestión de velocidad: depende del contexto, restricciones de memoria, estabilidad requerida y las características propias de los datos.



Casos de Uso Reales por Algoritmo



Bubble Sort

- Fines didácticos y demostración de conceptos
- Arreglos muy pequeños ($n < 20$)
- Datos casi completamente ordenados
- Detección de si un arreglo está ordenado



Insertion Sort

- Streaming de datos en tiempo real
- Listas cortas generadas dinámicamente
- Complemento de Quick Sort para subarreglos pequeños
- Sistemas embebidos con memoria limitada



Selection Sort

- Cuando las escrituras en memoria son costosas
- Sistemas con memoria flash o EEPROM
- Selección de los k menores/mayores elementos
- Arreglos pequeños con restricciones de swap

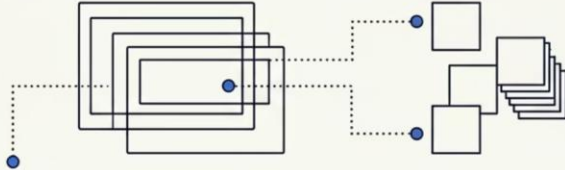


Quick Sort

- Ordenamiento general de grandes volúmenes
- Bases de datos relacionales y NoSQL
- Sistemas operativos (qsort en C)
- Motores de búsqueda y rankings en tiempo real

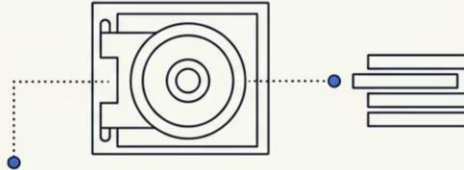
Aplicaciones en Sistemas Empresariales

1) E-Commerce



Ordenar productos por precio, relevancia, y calificación

2) Banca y Finanzas



Ordenar transacciones por fecha, monto, y prioridad de riesgo

3) Sistemas de Salud



Ordenar pacientes por urgencia, historial, y citas

4) Logística



Ordenar rutas por distancia, tiempo, y prioridad de entrega

Los algoritmos de ordenamiento son la columna vertebral invisible de los sistemas modernos. Desde el feed de tu red social hasta el sistema de pagos de un banco, el ordenamiento eficiente impacta directamente la experiencia del usuario y los costos operativos de la infraestructura.

Reflexión: Elige el Algoritmo Correcto

Analiza los siguientes escenarios y determina qué algoritmo de ordenamiento sería la mejor elección. Justifica tu respuesta considerando el tamaño de datos, la necesidad de estabilidad y las restricciones del sistema.

1

Escenario A

Un sistema de turnos médicos recibe nuevos pacientes uno por uno y debe mantener la lista ordenada por hora de llegada en tiempo real. El sistema tiene memoria muy limitada. ¿Qué algoritmo usarías?

2

Escenario B

Una plataforma de streaming necesita ordenar 50 millones de canciones por popularidad para generar rankings globales cada hora. La velocidad es crítica. ¿Cuál es tu elección?

3

Escenario C

Un microcontrolador IoT debe ordenar 10 lecturas de temperatura para encontrar la mediana. La memoria es extremadamente limitada y la velocidad no es prioritaria. ¿Qué usarías?



Soluciones Comentadas

Escenario A → Insertion Sort

Ideal para datos en streaming. Cada nuevo paciente se inserta en su posición correcta en $O(n)$ sin necesidad de re-ordenar todo el arreglo. Es estable y usa $O(1)$ de memoria adicional.

Escenario B → Quick Sort

Para 50 millones de registros, $O(n \log n)$ es la única opción viable. Quick Sort con pivote aleatorio evita el peor caso y en la práctica supera a otros algoritmos de su clase por sus constantes bajas.

Escenario C → Selection o Insertion Sort

Con solo 10 elementos, la diferencia de complejidad es insignificante. Ambos usan $O(1)$ de memoria. Selection Sort minimiza las escrituras (útil en flash memory); Insertion Sort es más simple de implementar.

Pon a Prueba Tu Conocimiento

Pregunta 1

¿Cuál es la complejidad en el **peor caso** de Quick Sort?

- A) $O(n \log n)$
- B) $O(n)$
- C) $O(n^2)$ 
- D) $O(\log n)$

Pregunta 2

¿Qué algoritmo minimiza el número de **intercambios** en memoria?

- A) Bubble Sort
- B) Quick Sort
- C) Insertion Sort
- D) Selection Sort 

Pregunta 3

¿Cuál de estos algoritmos es **estable**?

- A) Insertion Sort 
- B) Quick Sort
- C) Selection Sort
- D) Ninguno

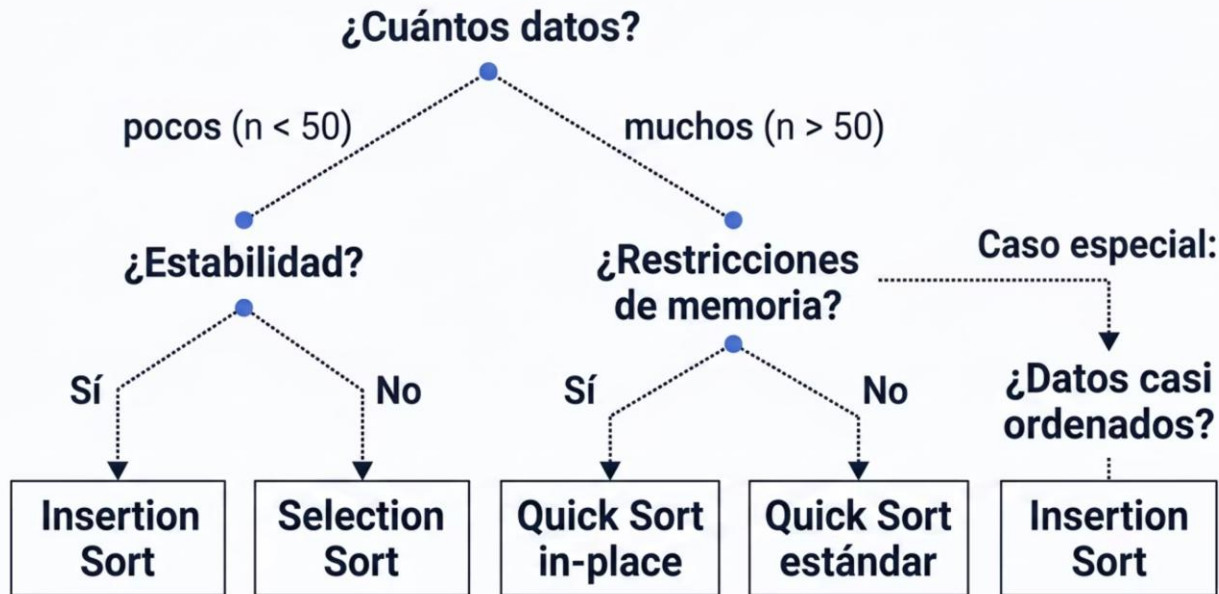
Pregunta 4

¿En qué caso Bubble Sort alcanza $O(n)$?

- A) Arreglo invertido
- B) Arreglo ya ordenado 
- C) Arreglo aleatorio
- D) Nunca

Guía de Selección de Algoritmos

Usa este mapa de decisión como referencia rápida al elegir un algoritmo de ordenamiento en tus proyectos. La clave está en evaluar el contexto completo, no solo la velocidad teórica.



Regla de oro: para uso general con n grande, Quick Sort es el punto de partida. Para datos pequeños o streaming, Insertion Sort. Para casos pedagógicos o depuración, Bubble Sort.

Conclusiones Clave

El análisis de eficiencia no es un ejercicio teórico: es la habilidad que distingue a un programador competente de un ingeniero de software profesional.

→ La notación Big O es tu brújula

Permite predecir el comportamiento de un algoritmo a cualquier escala, independientemente del hardware o lenguaje de programación.

→ $O(n \log n)$ es el estándar de excelencia

Quick Sort domina en la práctica. Conocer por qué —y cuándo falla— te permite tomar decisiones de diseño superiores en sistemas reales.

→ No existe el "mejor algoritmo universal"

Cada algoritmo tiene su contexto ideal. El tamaño de los datos, la memoria disponible y la necesidad de estabilidad son factores determinantes en la elección.

→ La eficiencia impacta el negocio

En sistemas con millones de usuarios, pasar de $O(n^2)$ a $O(n \log n)$ puede significar la diferencia entre una app que escala y una que colapsa bajo carga.